

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED

***CO5:** Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 10%

QUESTIONS: 4 Total Marks: 20 Duration: 60 Minutes Annexure A

Mock Interview

Code of Conduct:

I ____ A.Shiva Shankar Reddy (192425174)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ____ A.Shiva Shankar Reddy ____

Rubrics for Evaluation

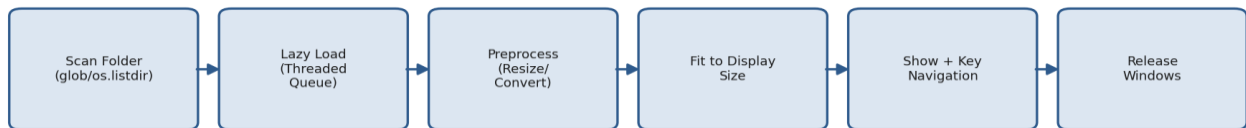
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks (100)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	

OpenCV Basics & Pipeline Thinking

1. Efficient Pipeline to Read, Process, and Display Multiple Images from a Folder

Interview Question: Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.

Pipeline: Efficient Multi-Image Read/Process/Display



Answer

I would structure this as a five-stage pipeline: folder scanning, lazy/threaded loading, preprocessing, display-fitting, and an interactive display loop, so that no stage blocks another and memory use stays bounded regardless of folder size.

Stage 1: Folder Scanning

Use `os.listdir()` or, more robustly, `glob.glob(folder + '/*.jpg')` combined with a whitelist of valid extensions (`.jpg`, `.png`, `.bmp`, `.tiff`) so non-image files are filtered out before any I/O is attempted. The file list is sorted (`sorted()`) for deterministic, repeatable ordering.

Stage 2: Lazy / Threaded Loading

Rather than calling `cv2.imread()` on every file up front — which for a large folder can exhaust RAM — I load images one at a time using a generator, or better, a background thread that pushes decoded frames into a bounded queue. `Queue(maxsize=5)` while the main thread displays the current image. This overlaps disk I/O and JPEG/PNG decoding with display time, which is the key efficiency gain.

Stage 3: Preprocessing

Each loaded image is validated (IF `img IS NULL` THEN skip and log a warning — this guards against corrupted files) and then passed through whatever processing the task needs, e.g., `cv2.cvtColor()` for color-space conversion, `cv2.GaussianBlur()` for denoising, or a custom filter. Keeping preprocessing modular (a single function) makes it easy to swap in different operations without touching the I/O logic.

Stage 4: Fit to Display Size

Images from a folder rarely share the same resolution, so before `cv2.imshow()` I resize each image to fit a fixed display window while preserving aspect ratio (compute `scale = min(maxW/w, maxH/h)`, then `cv2.resize()`). This keeps the viewing window stable rather than jumping in size between images.

Stage 5: Interactive Display Loop

`cv2.imshow()` shows the current image and `cv2.waitKey(0)` waits for a keypress: right-arrow/'n' advances to the next image, left-arrow/'p' goes back, 'q' quits. This gives the user control instead of an uncontrolled slideshow, and `cv2.destroyAllWindows()` is called once at the end to clean up GUI resources.

Design Justification

- Threaded prefetching overlaps slow disk/decode time with fast display time, so the pipeline is not I/O-bound.
- A bounded queue caps memory usage — only a handful of images are ever held in RAM at once, regardless of folder size.
- Validating every image (checking for None) prevents a single corrupted file from crashing the whole batch.
- Separating preprocessing into its own function keeps the pipeline modular and testable.

Potential Challenges

- Mixed image formats and color depths (8-bit vs 16-bit, grayscale vs color) needing normalization before display.
- Very large folders (tens of thousands of images) making even a file listing slow — mitigated with `os.scandir()` instead of `os.listdir()`.
- Corrupted or partially downloaded files causing `cv2.imread()` to return None silently.
- Memory spikes if preprocessing produces large intermediate arrays (e.g., high-resolution blurred copies).

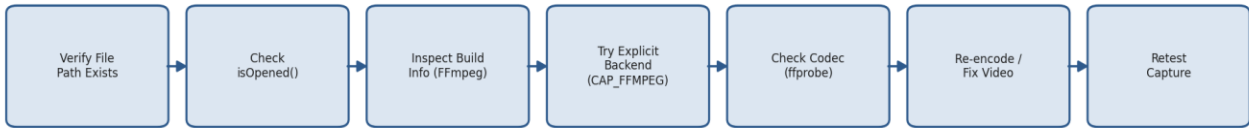
Optimizations

- Use `cv2.IMREAD_REDUCED_COLOR_2 / _4` flags to decode directly at a lower resolution when full resolution isn't needed, saving both decode time and memory.
- Cache generated thumbnails to disk on first pass so repeated viewing sessions are instant.
- Use a `multiprocessing.Pool` for CPU-heavy preprocessing (e.g., filtering) so multiple images are processed in parallel across cores while a single thread still owns the GUI display loop (OpenCV's HighGUI is not thread-safe for `imshow`).

2. Debugging a Video File That Is Not Loading Properly in OpenCV

Interview Question: A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.

Debugging Flow: Video Not Loading in OpenCV



Answer

I would treat this as a systematic elimination process, moving from the most common and easiest-to-check causes toward deeper codec/build issues, rather than guessing.

Possible Causes

Cause	Explanation
Incorrect file path	A relative path resolved from the wrong working directory, or a typo, silently makes VideoCapture fail with no error
OpenCV built without FFmpeg	The pip opencv-python wheel usually bundles FFmpeg, but custom/conda builds may lack video backend support entirely
Unsupported codec/container	Files encoded with codecs like HEVC/H.265 or unusual containers may not be decodable by the installed FFmpeg version
Corrupted or partially downloaded file	A truncated file often opens (isOpened() = True) but fails on the first read()
Wrong backend selected automatically	On some systems the default backend (e.g., CAP_MSMF on Windows) misbehaves for a given file; a different backend may work
Permission/access issues	The process lacks read permission on the file or the file is locked by another application

Structured Debugging Steps

```
STEP 1: Verify the path
ASSERT os.path.exists(video_path) == TRUE
-> if false: fix the path (use absolute path, check for typos)

STEP 2: Check whether the capture object opened
cap <- cv2.VideoCapture(video_path)
IF NOT cap.isOpened() THEN
    PRINT "VideoCapture failed to open the file"
    -> proceed to Step 3

STEP 3: Inspect OpenCV's build information
PRINT cv2.getBuildInformation()
-> check the 'Video I/O' section for 'FFMPEG: YES'
-> if FFMPEG is missing, reinstall: pip install opencv-python
    (avoid opencv-python-headless if GUI/video features are needed)

STEP 4: Try an explicit backend
cap <- cv2.VideoCapture(video_path, cv2.CAP_FFMPEG)
```

```
-> if this succeeds where the default failed, the issue was backend
selection

STEP 5: Inspect the codec with an external tool
RUN ffprobe video_path
-> note the codec_name (e.g., hevc, vp9, mpeg4)
-> cross-check against the FFmpeg version bundled with OpenCV

STEP 6: Re-encode to a known-good format
RUN ffmpeg -i input.mov -c:v libx264 -c:a aac output.mp4
-> retest cv2.VideoCapture('output.mp4')
-> if this now works, the original codec/container was the root cause

STEP 7: Confirm frame reads succeed
success, frame <- cap.read()
IF NOT success THEN
    PRINT "File opened but no frames could be decoded -> likely corrupted
file"
```

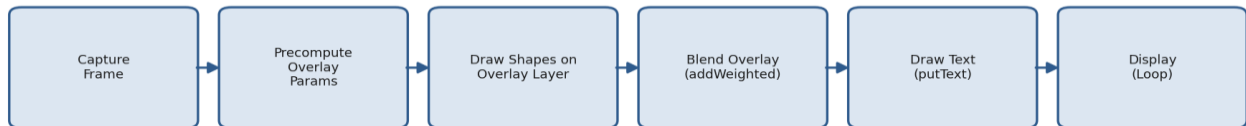
Why This Order

Checking the path and `isOpened()` first eliminates the two most frequent, trivial causes in seconds. Inspecting `cv2.getBuildInformation()` next rules out an environment/installation problem before spending time on the video file itself. Only after the environment is confirmed correct do I move to codec-specific diagnosis (`ffprobe`) and a re-encode test, which isolates whether the file itself is the problem. This ordering avoids wasted effort re-encoding a file when the real issue was simply a missing FFmpeg backend or a wrong path.

3. Overlaying Text and Shapes on a Live Video Feed

Interview Question: You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.

Pipeline: Overlay Text & Shapes on Live Video



Answer

The key efficiency principle is: precompute everything that does not change frame-to-frame outside the main loop, and only do per-frame work inside it. I would also separate solid annotations (text, opaque boxes) from semi-transparent shapes, since transparency requires an extra blending step.

Design

- Before the loop: open the camera with `cv2.VideoCapture(0)`; fix the font (`cv2.FONT_HERSHEY_SIMPLEX`), font scale, color, and thickness as constants; precompute any fixed shape coordinates (e.g., a corner logo box) once.
- Inside the loop: read a frame, draw solid shapes directly on it with `cv2.rectangle()` / `cv2.circle()` / `cv2.line()` / `cv2.polylines()`; for anything that needs to look semi-transparent (e.g., a shaded region-of-interest), draw it on a copy of the frame and blend with `cv2.addWeighted(overlay, alpha, frame, 1-alpha, 0, frame)`.
- Text is added last with `cv2.putText()`, since text is typically opaque and should sit on top of any shape layer.
- A live FPS counter (computed from `time.time()` deltas) is drawn using the same `cv2.putText()` call, giving real-time feedback on the overlay's own performance cost.

Key OpenCV Functions and Reasoning

Function	Purpose / Reasoning
<code>cv2.rectangle()</code> , <code>cv2.circle()</code> , <code>cv2.line()</code> , <code>cv2.polylines()</code>	Draw solid geometric shapes directly on the frame in-place — O(pixels drawn), negligible per-frame cost
<code>cv2.putText()</code>	Renders text; font/scale/thickness are precomputed constants so only the string content changes per frame
<code>cv2.getTextSize()</code>	Computed once (or only when the text changes) to correctly position/center text without recomputing layout every frame
<code>cv2.addWeighted()</code>	Efficiently blends a semi-transparent overlay layer with the base frame using a single vectorized operation instead of manual pixel looping
<code>cv2.rectangle(..., thickness=-1)</code>	A negative thickness fills the shape, useful for building the overlay layer for text backgrounds before blending

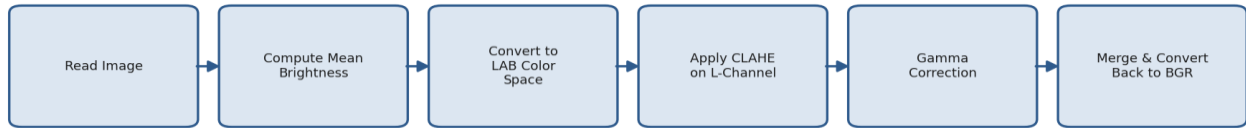
Efficiency Considerations

- Avoid creating a new overlay copy (`frame.copy()`) unless a transparent element is actually needed that frame — copying every frame unnecessarily wastes memory bandwidth.
- Keep all styling constants (fonts, colors, coordinates) defined once outside the WHILE loop rather than reconstructed each iteration.
- If overlay content changes only occasionally (e.g., a counter that updates once per second), redraw only when the value changes rather than recomputing text size every frame.
- For dynamic positions (e.g., text that must stay centered as frame size changes), compute position as a fraction of `frame.shape` so the overlay adapts without hardcoded pixel coordinates.

4. Handling an Image That Appears Too Dark When Read

Interview Question: An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

Pipeline: Fixing a Dark / Underexposed Image



Analysis of the Issue

A dark image is almost always a histogram problem: pixel intensities are clustered near 0 rather than spread across the full 0–255 range. This can arise from genuine underexposure at capture time (low ambient light, short exposure/high shutter speed, small aperture), from backlighting where the subject is dark against a bright background, or — less commonly — from a color-space/bit-depth mismatch on read (e.g., a 16-bit image read and displayed as if it were 8-bit, which crushes most values toward black). The first diagnostic step is to compute the mean pixel intensity (`cv2.mean(gray)`) and inspect the histogram (`cv2.calcHist()`) to confirm it is genuinely dark rather than a display artifact.

Proposed Pipeline Modification

Stage 1: Brightness Check

`meanVal <- cv2.mean(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY))`[0]. If `meanVal` is below a threshold (e.g., 90 on a 0–255 scale), the image is flagged as underexposed and routed through a correction branch.

Stage 2: Convert to LAB Color Space

`img_lab <- cv2.cvtColor(img, cv2.COLOR_BGR2LAB)`. Working in LAB rather than adjusting the B, G, R channels independently is important: LAB separates lightness (L) from color information (A, B), so brightening only the L channel improves visibility without shifting the image's color balance — a common artifact of naively equalizing each BGR channel separately.

Stage 3: Apply CLAHE to the L-Channel

`clahe <- cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8)); L2 <- clahe.apply(L)`. CLAHE (Contrast Limited Adaptive Histogram Equalization) is preferred over global `cv2.equalizeHist()` because it operates on small local tiles and clips the histogram before equalizing, which boosts local contrast in dark regions without over-amplifying noise or blowing out already-bright areas — a known weakness of global equalization on images with mixed lighting.

Stage 4: Gamma Correction

A power-law transform, $\text{output} = 255 * (\text{input}/255)^\gamma$ with $\gamma < 1$ (e.g., 0.5), is applied via a precomputed lookup table (`cv2.LUT()`) for efficiency. Gamma correction non-linearly boosts mid-tone and shadow brightness while compressing highlights less aggressively

than a simple linear brightness add (cv2.convertScaleAbs with a beta offset), giving a more natural-looking brightened image.

Stage 5: Merge and Convert Back

lab2 <- cv2.merge([L2, A, B]); result <- cv2.cvtColor(lab2, cv2.COLOR_LAB2BGR). The corrected lightness channel is recombined with the untouched color channels and converted back to BGR for display or downstream processing.

Justification

Technique	Why It Improves Visibility Correctly
Mean-brightness check	Automatically detects underexposure so correction is applied only when needed, avoiding unnecessary processing on well-exposed images
LAB + L-channel-only correction	Brightens the image without distorting hue/saturation, unlike per-channel BGR equalization
CLAHE (vs. global equalizeHist)	Improves local contrast in dark regions while limiting noise amplification and avoiding over-brightening already-lit areas
Gamma correction via LUT	Non-linearly lifts shadow/mid-tone detail with a natural look, and a precomputed LUT keeps the operation fast (O(256) lookup vs per-pixel math)